# LeetCode Practice

## 3 Easy: Greedy

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

**3**
**EASY**

**Patterns (in order)**

1. Greedy (3)

**How to use**

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

# Contents

**3 questions | tap a title to jump to it**


**Easy questions (3)** Go >

**Greedy (3)** Go >

E 1. Longest Palindrome LC 409

E 2. Lemonade Change LC 860

E 3. Apple Redistribution into Boxes LC 3074

## Easy questions (3)

EASY | TOPIC 1 OF 1

# Greedy

**3 Easy questions start on the next page**

**Before this:** this is the first topic
**After this:** this is the last topic

---

**IN THIS TOPIC**

**1.** Longest Palindrome LC 409
**2.** Lemonade Change LC 860
**3.** Apple Redistribution into Boxes LC 3074

EASY | Greedy | LeetCode #409 | 1 of 3 | 1/3

# Longest Palindrome

## QUESTION

Given a string s which consists of lowercase or uppercase letters, return the length of the longest palindrome that can be built with those letters.

Letters are case sensitive, for example, "Aa" is not considered a palindrome.

| Example 1 | |
|---|---|
| Input | `s = "abccccdd"` |
| Output | 7 |
| Explanation | *One longest palindrome that can be built is "dccaccd", whose length is 7.* |

| Example 2 | |
|---|---|
| Input | `s = "a"` |
| Output | 1 |
| Explanation | *The longest palindrome that can be built is "a", whose length is 1.* |

CONSTRAINTS

- 1 <= s.length <= 2000
- s consists of lowercase and/or uppercase English letters only.

## INTERVIEW APPROACH · STAR-LC

### PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Given a string of letters, find the length of the longest palindrome we can BUILD

from those characters — we rearrange them, we don't find a substring.

Letters are case-sensitive. Right?"

"A few quick questions:

Can we use every character? Yes — we pick the optimal subset.

Return the length, not the actual palindrome string? Yes."

"Let me walk: 'abccccdd' -> d×2, c×4, a×1, b×1.

Pairs: dd, cccc -> 6 chars. One leftover can be center -> total 7 ok

### PHASE 2 BRUTE FORCE

*"Let me start with brute force to lock in the logic."*

BRUTE FORCE · PSEUDO-CODE

```
for subset in subsets(letters):
    if it can be arranged as a palindrome (at
        most one letter with an odd count):
        best = max(best, candidate)
```

### PHASE 2 BRUTE FORCE

| TIME | SPACE |
|---|---|
| O(2^n * n) | O(n) |

WHY IT IS SLOW

Tries every subset. Count the letters: every pair of a letter can be used, plus one single leftover letter in the middle: O(n).

*"Should I go ahead and optimize?"*

### PHASE 3 OPTIMIZE

The bottleneck is the separate Counter build and scan steps.

The key insight: we only need to know which characters have ODD frequency.

Toggle each char in a set — present means odd count so far, absent means even.

Final answer = len(s) - len(odd_chars) + (1 if odd_chars else 0).

**Time:** O(n). Space: O(1) — at most 52 entries.

Does that make sense before I code it?

### PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _409_LongestPalindrome:
    def longestPalindrome(self, s):
        odd_chars = set()
        for ch in s:
            if ch in odd_chars:
                odd_chars.discard(ch)
            else:
                odd_chars.add(ch)
        return len(s) - len(odd_chars) + (1 if odd_chars e
lse 0)
```

### PHASE 5 TEST & DEBUG

*"Let me trace the greedy choice and show why that choice can't hurt the optimal result."*

'abccccdd':

a->{a}, b->{a,b}, c->{a,b,c}, c->{a,b}, c->{a,b,c}, c->{a,b}, d->{a,b,d}, d->{a,b}.

odd_chars={a,b}. len(s)=8. 8-2+1=7 ok

'a':

odd_chars={a}. 1-1+1=1 ok

'aa':

odd_chars={}. 2-0+0=2 ok

The +1 only fires when there are leftover characters to place at the center

## PHASE 6 COMPLEXITY & FOLLOW-UP

| TIME | SPACE |
| --- | --- |
| **O(n)** | **O(1)** |
| one pass | set has at most 52 entries |

The set-toggle is slightly more elegant than Counter since it avoids the intermediate dict.

Both approaches have identical complexity.

**Follow-up:** Palindrome Pairs (#336) — build palindromes from word concatenations.

**Follow-up:** if we must also return the actual palindrome, assign pairs to both ends

and place one odd character in the middle.

IN YOUR APP | Expose | Editorial | Community

EASY | Greedy | LeetCode #860 | 2 of 3 | 2/3

# Lemonade Change

## QUESTION

At a lemonade stand, each lemonade costs $5. Customers are standing in a queue to buy from you and order one at a time (in the order specified by bills). Each customer will only buy one lemonade and pay with either a $5, $10, or $20 bill. You must provide the correct change to each customer so that the net transaction is that the customer pays $5.

Note that you do not have any change in hand at first.

Given an integer array bills where bills[i] is the bill the ith customer pays, return true if you can provide every customer with the correct change, or false otherwise.

**Example 1**

| | |
|---|---|
| Input | `bills = [5,5,5,10,20]` |
| Output | `true` |
| Explanation | *From the first 3 customers, we collect three $5 bills in order.*<br>*From the fourth customer, we collect a $10 bill and give back a $5.*<br>*From the fifth customer, we give a $10 bill and a $5 bill.*<br>*Since all customers got correct change, we output true.* |

**Example 2**

| | |
|---|---|
| Input | `bills = [5,5,10,10,20]` |
| Output | `false` |
| Explanation | *From the first two customers in order, we collect two $5 bills.*<br>*For the next two customers in order, we collect a $10 bill and give back a $5 bill.*<br>*For the last customer, we can not give the change of $15 back because we only have two $10 bills.*<br>*Since not every customer received the correct change, the answer is false.* |

CONSTRAINTS

- $1 <= bills.length <= 10^5$
- bills[i] is either 5, 10, or 20.

## INTERVIEW APPROACH · STAR-LC

### PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Each lemonade costs $5. Customers pay with $5, $10, or $20 bills, in the order given.

Determine if you can provide correct change to every customer. Right?"

"A few quick questions:

### PHASE 1 CLARIFY

Start with no change in hand? Yes — must make change purely from bills collected so far."

"Let me walk: bills=[5,5,5,10,20] -> true (last customer's $20 gets one $5+one $10 back) ok

### PHASE 2 BRUTE FORCE

*"Let me start with brute force to lock in the logic."*

BRUTE FORCE · PSEUDO-CODE

```
keep every bill received in a list
for customer in customers:
    find a set of bills in the list that makes
        the change by scanning
```

| TIME | SPACE |
|---|---|
| O(n^2) | O(n) |

WHY IT IS SLOW

Searches the bill list for every customer. Only $5 and $10 bills matter, so counting them gives O(1) per customer: give a 10 and a 5, or three 5s, for a $20; one 5 for a $10.

*"Should I go ahead and optimize?"*

### PHASE 3 OPTIMIZE

*"Greedy with three counters — O(n). Walk-through:"*

five=0, ten=0, twenty=0 (bills on hand).

**For each bill:**

$5: five++. No change needed.

$10: need $5 change. If five==0: return False. five--; ten++.

$20: prefer to give $10+$5 change (uses fewer $5 bills, preserving them for future).

If ten>0 and five>0: ten--; five--; twenty++.

Elif five>=3: five-=3; twenty++.

Else: return False.

**WHY prefer $10+$5 over $5+$5+$5:** $10 bills are useless for anything except $20 change.

$5 bills are versatile (used for $10 and $20 change). Save them when possible.

**Time:** O(n). Space: O(1).

### PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _860_LemonadeChange:
    def lemonadeChange(self, bills):
        five_count = ten_count = 0
        for bill in bills:
            if bill == 5:
                five_count += 1
            elif bill == 10:
```

```
            if five_count == 0:
                return False
            five_count -= 1
            ten_count += 1
        else:
            if ten_count > 0 and five_count > 0:
                ten_count -= 1
                five_count -= 1
            elif five_count >= 3:
                five_count -= 3
            else:
                return False
    return True
```

## PHASE 5 TEST & DEBUG

*"Let me trace lemonadeChange([5,5,5,10,20])."*

Track: five=0, ten=0

Receive 5: no change needed. five=1

Receive 5: five=2

Receive 5: five=3

Receive 10: give $5 change. five=2, ten=1

Receive 20: prefer give 10+5 (saves 5s). ten=0, five=1. True ok

Return True ok (can give change to all)

**WHY prefer 10+5 for $20 change:** $10 bills are less flexible (can't use for $10 customers).

Edge: first customer pays $10 with no $5s in hand -> return False immediately.

## PHASE 6 COMPLEXITY & FOLLOW-UP

| TIME | SPACE |
|---|---|
| O(n) | O(1) |

**Follow-up:** Boats to Save People (#881) — different greedy, two-pointer on sorted weights.

**Follow-up:** if bills could also be $1 or $50, the greedy choice logic would need extending.

EASY | Greedy | LeetCode #3074 | 3 of 3 | 3/3

# Apple Redistribution into Boxes

## QUESTION

You are given an array apple of size n and an array capacity of size m.

There are n packs where the ith pack contains apple[i] apples. There are m boxes as well, and the ith box has a capacity of capacity[i] apples.

Return the minimum number of boxes you need to select to redistribute these n packs of apples into boxes.

Note that, apples from the same pack can be distributed into different boxes.

**Example 1**

| | |
|---|---|
| Input | `apple = [1,3,2], capacity = [4,3,1,5,2]` |
| Output | `2` |
| Explanation | *We will use boxes with capacities 4 and 5. It is possible to distribute the apples as the total capacity is greater than or equal to the total number of apples.* |

**Example 2**

| | |
|---|---|
| Input | `apple = [5,5,5], capacity = [2,4,2,7]` |
| Output | `4` |
| Explanation | *We will need to use all the boxes.* |

CONSTRAINTS

- 1 <= n == apple.length <= 50
- 1 <= m == capacity.length <= 50
- 1 <= apple[i], capacity[i] <= 50
- The input is generated such that it's possible to redistribute packs of apples into boxes.

## INTERVIEW APPROACH · STAR-LC

### PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

We have apple[i] apples in box i and capacity[j] capacity in box j.

Minimum boxes needed to hold all apples; boxes chosen by largest capacity first. Right?"

"A few quick questions:

Sort capacity descending; greedily fill until total apples covered."

"Let me walk: apple=[1,3,2], capacity=[4,3,1,5,2] -> total=6. sorted_cap=[5,4,3,2,1].

5>=6? No. 5+4=9>=6 -> 2 boxes ok

### PHASE 2 BRUTE FORCE

*"Let me start with brute force to lock in the logic."*

BRUTE FORCE · PSEUDO-CODE

```
for subset in subsets(boxes):
    if their capacity covers the total apples:
        best = min(best, subset_size)
```

| TIME | SPACE |
|---|---|
| O(2^m) | O(1) |

WHY IT IS SLOW

Tries every subset. Sort the capacities in descending order and take boxes until the capacity covers the total: O(m log m).

*"Should I go ahead and optimize?"*

### PHASE 3 OPTIMIZE

*"Greedy: sum n largest apples, find min boxes — O(n log n). Walk-through:"*

Sort baskets in descending order. Take the first n baskets (n = neededApples count).

Wait — actual approach:

total_apples needed = neededApples. Sort capacity descending.

**Greedily fill the largest boxes first:**

Accumulate sum. Stop when sum >= neededApples. Count boxes used.

**WHY largest first:** minimizes the number of boxes (maximize apples per box used).

**Time:** O(m log m) where m = len(capacity). Space: O(1).

### PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _3074_AppleRedistributionIntoBoxes:
    def minimumBoxes(self, apple, capacity):
        total_apples = sum(apple)
        capacity.sort(reverse=True)
        accumulated = 0
        for i, cap in enumerate(capacity):
            accumulated += cap
            if accumulated >= total_apples:
                return i + 1
        return len(capacity)
```

### PHASE 5 TEST & DEBUG

*"Let me trace the greedy choice and show why that choice can't hurt the optimal result."*

apple=[1,3,2], capacity=[4,3,1,5,2]: total=6. sorted=[5,4,3,2,1].

i=0(5): acc=5<6. i=1(4): acc=9>=6 -> return 2 ok

apple=[5,5,5], capacity=[10,5]: total=15. sorted=[10,5].
acc=10<15. acc=15>=15 -> return 2 ok

## PHASE 5 TEST & DEBUG

Greedy picks largest boxes first to minimize count

## PHASE 6 COMPLEXITY & FOLLOW-UP

| TIME | SPACE |
| --- | --- |
| **O(n log n)**<br>sort dominates | **O(1)** |

**Follow-up:** if boxes have a fixed cost, minimise total cost instead of count.

**Follow-up:** Assign Cookies (#455) — similar greedy with sorting.

IN YOUR APP | Expose | Editorial | Community